

GradeUp

Relazione tecnica della piattaforma web

Programmazione Web e Mobile

Anno Accademico 2025-2026

Professore: Luca Cruciata

Autore: Gabriele Iovino

1. Introduzione

GradeUp è una piattaforma pensata per accompagnare lo studente lungo tutta la sua carriera universitaria, dall'immatricolazione fino alla verbalizzazione degli esami, e per dare al docente gli strumenti per gestire i propri corsi e il materiale didattico.

L'idea di fondo è raccogliere in un unico posto attività quotidiane della vita universitaria che normalmente sono sparse tra pagine diverse o difficilmente raggiungibili.

Una scelta che vale la pena anticipare subito è che la stessa codebase supporta sia desktop che mobile. Questo è possibile perché il frontend è costruito con Angular e Ionic, una combinazione che permette di scrivere l'interfaccia una volta sola e di adattarla ai due contesti senza duplicare il lavoro o ricorrere a css over-engineerizzati.

L'architettura generale

L'architettura è quella di una classica piattaforma web, organizzata su 3 livelli.

In primis, il client, l'applicazione Angular con cui l'utente interagisce.

Il secondo è il backend vero e proprio, Node con Express che espone le API REST;

Il terzo è il DB, con PostgreSQL come database relazionale e un object storage per i file caricati dagli utenti (hostato su Supabase).

Nota: durante la fase di progettazione avevo previsto l'uso di Nginx come gateway su docker e Redis per la gestione di sessione e notifiche; durante lo sviluppo ho optato per una soluzione meno onerosa ma che mantenesse le stesse funzionalità. Anche l'ipotesi di MinIO come object storage è stata sostituita per comodità con i bucket S3 forniti direttamente da Supabase.

Come è organizzato il repository

Il codice applicativo vive sotto la cartella GradeUp ed è diviso in tre parti indipendenti, ciascuna con le proprie dipendenze: il backend, che contiene l'API Express; il frontend, che contiene l'applicazione Angular; e una terza cartella, shared, che merita una spiegazione a parte perché è il punto di incontro tra le altre due.

Ho optato per questa struttura per poter gestire comodamente modelli e classi condivise tra BE e FE (DRY!)

2. Le scelte di sviluppo e le tecnologie

Validazione dei dati centralizzata

La cartella shared contiene un pacchetto TypeScript con un'unica dipendenza, la libreria di validazione Zod, che descrive gli schemi del dominio e i tipi che ne derivano.

Sia il BE che il FE importano questo pacchetto: lo schema di validazione di una richiesta di login (LoginDtoSchema in auth.ts), per fare un esempio concreto, è scritto una volta sola e usato sia dal controller che riceve i dati, sia dal form che li raccoglie. Tolta la comodità di sviluppo, questo permette ad esempio di appoggiarmi a un unico file TS ([enums.ts](#)) per creare “domini” personalizzati per le colonne a DB (ad esempio gli stati possibili di un esame)

PostgreSQL senza un ORM

Il backend dialoga con PostgreSQL attraverso query SQL scritte a mano e parametrizzate, senza interporre un ORM. E' una scelta deliberata: dato che la descrizione dei dati vive già negli schemi condivisi, introdurre un ORM significherebbe creare una seconda descrizione da tenere allineata alla prima, con il rischio costante che le due si allontanino. Mantenere SQL nativo, invece, rende le query leggibili a chiunque conosca Postgres e lascia il pieno controllo su cosa viene effettivamente eseguito.

Angular 21

Il frontend usa Angular nella sua versione più recente, in modalità a componenti autonomi: niente moduli da configurare, stato gestito con i signals e una strategia di aggiornamento della vista efficiente applicata in modo uniforme.

Sopra Angular si appoggiano Ionic, che fornisce la struttura e i componenti dell'interfaccia adatti sia al web sia al mobile, e Bootstrap, usato per la griglia e per le classi di utilità.

Autenticazione con token

L'accesso degli utenti è gestito in modo classico con token firmati, senza che il server conservi uno stato di sessione.

La prossima sezione entra nel merito;

3. La gestione dell'autenticazione

L'autenticazione e controlli sulle autorizzazioni seguono uno schema convenzionale: token auth emanato dal backend, che viene salvato in sessione dal browser e usato nelle chiamate API per validare la chiamata e gestire i permessi.

Entità anagrafica / utenti

Alla base c'è una tabella anagrafica comune che raccoglie i dati di ogni persona. Da questa si diramano due tabelle collegate, una per gli studenti e una per i docenti, in rapporto uno a uno con l'anagrafica (001_initial_schema.sql).

Un utente è studente oppure docente a seconda di quale delle due righe esiste per lui.

Il ruolo non è quindi un campo memorizzato sull'anagrafica, ma una conseguenza dei dati che viene dedotta al momento dell'accesso: questo tiene l'anagrafica pulita ed evita di dover mantenere coerente un'informazione duplicata.

Lato server: emissione e verifica

La registrazione valida i dati ricevuti, controlla che l'email non sia già presente, cifra la password con bcrypt (processo di hash irreversibile) prima di salvarla e crea sia la riga anagrafica sia quella dello studente, restituendo infine un token. Va notato che la registrazione pubblica crea soltanto studenti: per i docenti non è previsto un percorso di iscrizione autonomo. L'accesso, a sua volta, verifica le credenziali, controlla la password, determina il ruolo cercando l'utente tra gli studenti o tra i docenti e restituisce il token insieme ai dati dell'utente, naturalmente senza la password (register e login in auth.controller.ts).

Il token è un JWT firmato con scadenza a sette giorni e contiene il minimo indispensabile: l'identificativo dell'utente e il suo ruolo. La protezione delle rotte si appoggia a due controlli successivi: il primo legge il token dall'intestazione della richiesta e lo verifica, il secondo controlla che il ruolo corrisponda a quello richiesto dalla specifica rotta (authenticate e requireRole in auth.middleware.ts). Esistono anche gli endpoint per il recupero della password (forgotPassword e resetPassword in auth.controller.ts), ma non sono ancora operativi perché manca il collegamento all'invio delle email (servirebbe un mail server): la richiesta di recupero risponde in modo volutamente neutro, per non rivelare se un indirizzo sia registrato o meno.

Sessione, interceptor e guards

Nel frontend l'autenticazione ruota attorno a un servizio centrale, basato sui signal (AuthService in auth.service.ts), che custodisce la sessione e ne espone in lettura le informazioni utili: l'utente corrente, il suo ruolo, il token e l'indicazione se sia autenticato. La sessione viene conservata nel browser in modo da sopravvivere ai ricaricamenti, con l'accortezza di proteggere ogni accesso allo storage, dato che l'applicazione viene resa anche lato server, dove quegli strumenti del browser non esistono.

è doveroso fare riferimento ad altri 2 meccanismi. Il primo è un interceptor che, quando è presente un token e la chiamata è diretta alle API, aggiunge automaticamente l'intestazione

di autorizzazione: in questo modo nessuna pagina deve preoccuparsi di allegare il token a mano.

Il secondo è l'insieme di guards delle routes (`guest.guard.ts`, `student.guard.ts` e `teacher.guard.ts`), funzioni semplici che leggono lo stato di autenticazione e decidono se lasciar passare l'utente o reindirizzarlo. Una guard impedisce a chi è già connesso di tornare alle pagine pubbliche; le altre due proteggono rispettivamente l'area studente e l'area docente, rimandando al login chi non è autenticato e alla propria home chi tenta di entrare nell'area dell'altro ruolo.

4. Il backend

Il backend è scritto in TypeScript e si basa su Express.

Per parlare con il database usa un pool di connessioni a Postgres (`db` in `client.ts`);

Per l'autenticazione si appoggia a `bcrypt` e ai token JWT; per la validazione dei dati impiega `Zod`.

A completare il quadro ci sono gli strumenti ormai consueti per la sicurezza degli header (`helmet`), la compressione delle risposte (`compression`) e la registrazione delle richieste nei log (`morgan`)

Struttura MVC

Il backend non segue il classico pattern MVC.

A un controller corrisponde sempre una entità del DB, le query avvengono nella quasi totalità dei casi direttamente sui metodi del controller; questa è stata una scelta deliberata per evitare blotcode e wiring inutili tra classi.

L'assemblaggio avviene in un punto unico (`app.ts`), dove vengono montati nell'ordine i vari middleware e poi i gruppi di rotte suddivisi per area: autenticazione, utenti, dati dell'utente corrente, notifiche, corsi di laurea e corsi.

Le rotte che riguardano l'utente connesso sono raccolte sotto un unico prefisso (`me.routes.ts`), a cui viene applicata per prima cosa la verifica del token e poi, dove serve distinguere studente da docente, il controllo del ruolo. E proprio qui che la convenzione si vede meglio: le funzionalità dello studente e quelle del docente sono affidate a controller distinti, ciascuno responsabile della propria area.

La gestione degli errori

Per gli errori previsti, quelli che ci si aspetta durante il normale funzionamento, come una validazione fallita o una risorsa non trovata, esiste una classe di errore applicativo dedicata, accompagnata da un gestore centrale che la traduce in una risposta coerente in formato JSON (`AppError` e `errorHandler` in `error-handler.ts`). Lo stesso gestore intercetta anche gli errori di validazione che dovessero sfuggire, restituendo il dettaglio dei campi non corretti. Tutto ciò che invece non è previsto diventa un errore generico, registrato nei log del server ma non esposto al client.

Poiché i controller sono asincroni, un piccolo involucro (`asyncHandler` in `async-handler.ts`) si occupa di inoltrare al gestore centrale eventuali errori, in modo che nessuno resti silenzioso.

La validazione dei dati in ingresso

La validazione non è affidata a un controllo generico applicato a tutto, ma viene richiamata esplicitamente dentro ciascun controller, sullo schema condiviso che corrisponde a quella richiesta. Così facendo la regola di validazione resta vicina al punto in cui i dati vengono usati, e leggendo il controller si capisce immediatamente cosa ci si aspetta di ricevere.

5. Il frontend e l'architettura Angular

I principi adottati

Per la scrittura del codice lato client mi sono basato su alcuni principi implementativi, ad esempio:

- Lo stato locale è gestito con i signals e i valori derivati con funzioni calcolate; le dipendenze si ottengono tramite iniezione esplicita.
- Nei template si usa il controllo di flusso nativo della nuova sintassi, non le vecchie direttive (niente `*ngif`, `*ngfor` o simili).
- I form sono sempre di tipo reattivo.
- Template, stile e codice stanno in file separati: il foglio di stile di una pagina si aggiunge solo se servono davvero degli stili, la maggior parte delle pagine non ne ha bisogno grazie a bootstrap.

Come sono organizzate le cartelle

Il codice è organizzato in base al ruolo che ricopre, non disperso per funzionalità. C'è una cartella per i file lib (core), che raccoglie i servizi condivisi, le guards, gli interceptors, i validatori e le costanti con gli indirizzi delle API, suddivisi a loro volta in sottocartelle tematiche. C'è una cartella per i componenti riusabili e trasversali, come la struttura dell'area autenticata e gli elementi che mostrano gli stati di assenza dati o di errore.

E c'è infine la cartella delle pagine vere e proprie, raggruppate per area: quelle pubbliche, quelle dello studente, quelle del docente e le poche pagine identiche per entrambi i ruoli.

Per evitare i percorsi relativi a cascata, gli import si appoggiano ad alias configurati nel progetto, (definiti in `tsconfig.json`; es: `$core`, `$components`) che indicano in modo breve e stabile le quattro zone principali del codice, compreso il collegamento ai tipi del pacchetto condiviso.

I services e la gestione dello stato

Ogni area funzionale ha il proprio servizio, che racchiude le chiamate API e, dove serve, costruisce i modelli di dati che la pagina mostra (il `typesafe` è sempre mantenuto).

Gli indirizzi delle API non sono sparsi tra i servizi, ma raccolti in un unico file contenente tutti gli URI (`api.consts.ts`) suddiviso per area, così che modificare un percorso resti un'operazione da fare in un solo punto. Lo stato delle pagine segue uno schema ricorrente e leggibile: a ogni sezione che carica dati corrisponde uno stato con tre possibilità, ovvero caricamento in corso, errore con possibilità di riprovare, oppure dati pronti con l'eventuale caso di assenza di contenuti (gestito a livello visivo dal componente `error-state.ts`). Ogni sezione ha uno stato indipendente, quindi una parte in errore non trascina con se le altre.

I form e la validazione condivisa

I form sono di tipo reattivo e la loro validazione riutilizza gli stessi schemi del backend grazie a un adattatore: una piccola funzione (`zodValidator` in `zod.validator.ts`) trasforma uno schema condiviso in un validatore Angular, eseguendo il controllo sul valore del campo in tempo reale ed esponendo il messaggio in caso di errore. In questo modo una regola come la lunghezza minima della password è scritta una volta sola e vale identica sul client e sul server, mentre i messaggi tradotti in italiano restano definiti vicino alle pagine.

Interceptors

Il frontend registra due HTTP interceptor, funzioni che intercettano ogni richiesta HTTP in uscita prima che parta verso il server. Sono configurati in `app.config.ts`, tramite `provideHttpClient(withInterceptors([...]))`.

Il primo è l'`authInterceptor` (`auth.interceptor.ts`): quando esiste un token di sessione e la richiesta è diretta alle API (URL che inizia con `/api/`), aggiunge l'header `Authorization: Bearer <token>`. In questo modo l'autenticazione viaggia in automatico su ogni chiamata e nessun componente deve gestirla a mano.

Il secondo è il `loadingInterceptor` (`loading.interceptor.ts`): incrementa un contatore all'avvio di ogni richiesta e lo decrementa al suo termine. Finché il contatore è maggiore di zero, resta visibile la `ion-progress-bar` globale in cima all'applicazione. È l'unico indicatore di caricamento dell'app: si è scelto un progress bar centralizzato invece di spinner separati in ogni componente.

Lo stile e l'interfaccia

Alcuni schemi visivi ricorrono di frequente, pensati per non duplicare il codice. Le liste fitte di dati, come il catalogo o il libretto, usano un'intestazione di colonne visibile solo su schermo grande affiancata a un elenco: in questo modo le righe risultano allineate in modo ordinato sul desktop e si impilano come schede sul mobile, senza ricorrere a vere e proprie tabelle. Gli stati di assenza dati e di errore sono affidati a due componenti riusabili (`empty-state.ts` e `error-state.ts`). Il riscontro alle azioni dell'utente passa per un servizio di notifiche temporanee con un punto di visualizzazione unico a livello di applicazione (`toast.service.ts`).

6. Le funzionalità

Area studente

Immatricolazione

L'immatricolazione parte dalla scelta di un corso di laurea, attinto dall'elenco anagrafico dei corsi disponibili (`degrees.controller.ts`).

La logica (`createMatriculation` in `me.controller.ts`) verifica che il corso di laurea esista, che lo studente non risulti già immatricolato con uno stato `active` o `pending`, e genera quindi una matricola progressiva calcolata come $\text{MAX}(\text{matriculation_code}) + 1$ a partire da una base d'anno; la riga viene inserita con stato `active` e data corrente.

Il vincolo "una sola immatricolazione viva" non è demandato a un trigger sul database ma applicato esplicitamente nel controller.

Catalogo e iscrizione ai corsi

Il catalogo (`catalog` in `courses.controller.ts`) restituisce tutti gli insegnamenti con il relativo docente, i CFU e il numero di iscritti, arricchiti da due flag calcolati rispetto allo studente corrente: `inMyPlan`, se l'insegnamento appartiene al suo piano di studi, e `alreadyRegistered`, se è già iscritto. L'iscrizione vera e propria (`createRegistration` in `me.controller.ts`) è subordinata a un'immatricolazione `active` sul corso di laurea dell'insegnamento, rifiuta i duplicati e rispetta l'eventuale `max_students` contando le iscrizioni esistenti. La disiscrizione (`deleteRegistration`) è pensata per non lasciare prenotazioni orfane: prima ritira le prenotazioni `scheduled` agli appelli di quel corso, poi rimuove la riga di `registration`.

Prenotazione e ritiro dagli appelli

La prenotazione (`createEnrollment` in `me.controller.ts`) impone una catena di precondizioni: lo studente deve essere iscritto al corso, l'appello deve essere futuro e non deve esistere già una prenotazione `scheduled` per lo stesso appello. La prenotazione nasce con stato `scheduled`; il ritiro (`withdrawEnrollment`) non cancella la riga ma la porta a `withdrawn` registrando la data, così da preservare lo storico. La pagina degli appelli disponibili (`availableExams` in `student-dashboard.controller.ts`) mostra solo gli appelli futuri entro novanta giorni dei corsi a cui lo studente è iscritto, riportando per ciascuno il proprio stato.

Piano di studi

Il piano (`studyPlan` in `student-dashboard.controller.ts`) si costruisce a partire dalla tabella `study_plan` del corso di laurea attivo e, per ogni insegnamento previsto, deriva uno stato leggibile correlando i dati: `passed` se esiste una prenotazione superata con voto, `in_progress` se esiste un'iscrizione al corso, altrimenti `todo`. I CFU di ogni insegnamento sono recuperati dal corso più recente che eroga quella materia, e sommati per ottenere il totale del piano e quelli già acquisiti.

Libretto e medie

Il libretto (`transcript` in `student-dashboard.controller.ts`) raccoglie tutte le prenotazioni con stato `passed`, ciascuna con voto, CFU e dati dell'appello. Su questa base calcola i CFU acquisiti, il totale previsto dal piano e due medie distinte: quella aritmetica e quella ponderata sui CFU, arrotondate lato server. Tenere il calcolo nel backend evita che client diversi producano numeri diversi sullo stesso dato.

Cruscotto studente

La home dello studente è alimentata da più endpoint di sola lettura (student-dashboard.controller.ts): il riepilogo di carriera (careerSummary) seleziona l'immatricolazione attiva, o la più recente, e ne traduce il tipo di laurea in un'etichetta leggibile; i prossimi appelli (upcomingExams) elencano le prenotazioni scheduled ordinate per data; il progresso in crediti (cfuProgress) confronta i CFU acquisiti con quelli pianificati e affianca la media. Ogni riquadro ha il proprio endpoint, in linea con la gestione a stati indipendenti descritta per il frontend.

Profilo e cambio password

La lettura del profilo (getMe) e l'aggiornamento dei dati anagrafici modificabili (updateProfile) lavorano sulla tabella app_user.

Il cambio password (changePassword in me.controller.ts) verifica la password attuale con bcrypt.compare, rifiuta che la nuova coincida con la vecchia e salva il nuovo hash: la password in chiaro non transita mai oltre il confronto e non viene mai restituita.

Area docente

I corsi del docente

L'elenco dei corsi (listCourses e currentCourses in teacher-dashboard.controller.ts) è sempre filtrato per id_teacher, così un docente vede solo i propri insegnamenti, con il numero di iscritti e un flag che segnala i corsi conclusi in base alla data di fine. Il dettaglio (courseDetail) aggiunge l'elenco degli studenti iscritti con la loro matricola: anche qui l'appartenenza del corso al docente è verificata nella WHERE, non data per scontata.

Gestione degli appelli

La creazione di un appello (create in exam.controller.ts) richiede che il corso sia del docente e che la data sia futura. La modifica (update) è bloccata se esistono prenotazioni scheduled, per non spostare un appello sotto i piedi degli studenti già iscritti; l'annullamento (cancel) è impedito se l'appello è già stato verbalizzato e, quando ammesso, cancella in transazione sia le prenotazioni sia l'appello. Lo stato dell'appello — in_programma, da_verbalizzare, verbalizzato — non è memorizzato ma derivato in SQL dalla data e dalla presenza di prenotazioni ancora aperte.

Verbalizzazione degli esiti

La schermata di verbalizzazione (grading in exam.controller.ts) restituisce l'elenco dei prenotati con i conteggi di righe ancora da valutare e già chiuse. Il salvataggio (saveGrading) valida ogni riga lato server — lo stato deve appartenere all'insieme passed/failed/absent/withdrawn e il voto è obbligatorio e compreso tra 18 e 30 solo per gli esiti superati — e ammette la scrittura unicamente sulle prenotazioni ancora scheduled di quell'appello. Le modifiche vengono applicate dentro una transazione, così che la verbalizzazione sia tutto-o-niente.

Materiale didattico

Il caricamento del materiale (upload in archive.controller.ts) controlla il tipo MIME contro una allow-list, sanifica il nome del file e genera una chiave d'oggetto univoca con randomUUID. La sequenza è studiata per non lasciare incoerenze: il file viene prima caricato sul bucket (storage.service.ts), poi in un'unica transazione si creano o aggiornano la riga di archive, quella di file e quella di courseware; se la transazione fallisce, oltre al rollback si rimuove anche l'oggetto già caricato, per non lasciare file orfani. La rinomina e la rimozione operano solo su materiale di un corso del docente; il download non espone mai il bucket ma restituisce un link firmato a scadenza breve.

Cruscotto docente

La home del docente (`teacher-dashboard.controller.ts`) riunisce i corsi del periodo corrente, i prossimi appelli con il numero di prenotati e, soprattutto, gli appelli "da verbalizzare" (`examsToGrade`): quelli ormai passati che hanno ancora prenotazioni `scheduled`, ovvero il lavoro di verbalizzazione rimasto in sospeso.

Funzionalità trasversali

Notifiche. Le notifiche (`notifications.controller.ts`) sono per-utente e persistite a database: l'elenco (`listMine`) le restituisce ordinate per data con il flag di lettura e una severità, mentre `markRead` e `markAllRead` aggiornano lo stato di lettura sempre limitatamente all'utente autenticato. Si tratta di notifiche applicative consultate alla lettura, non di un canale push in tempo reale.

Object storage e gestione dei file

L'accesso ai file è incapsulato in un unico servizio (`storage.service.ts`) che fa da adattatore verso lo storage di Supabase ed espone tre sole operazioni: caricamento senza sovrascrittura, rimozione e generazione di URL firmati. Il principio è la separazione tra metadati e byte: il database conserva soltanto la chiave d'oggetto e gli attributi del file, mentre il contenuto vive nel bucket e viene servito esclusivamente tramite link temporanei, così che nessuna risorsa sia pubblicamente raggiungibile.